

GitCompass AI Engine - Algorithms & Techniques

This document explains the algorithms and techniques used in the GitCompass AI engine for each major task.

1. Resume Processing (Text Extraction)

Algorithm: Document Parsing using **PDFMiner** and **python-docx**

How it works:

- The uploaded resume file is detected by extension (**.pdf**, **.docx**, **.doc**)
- **PDFs:** **pdfminer.high_level.extract_text()** converts the PDF binary stream into raw text. PDFMiner works by interpreting the PDF page layout, extracting character-level data, and reconstructing lines/paragraphs
- **DOCX:** **python-docx** reads the **.docx** XML structure and concatenates all paragraph text
- **DOC/other:** Falls back to UTF-8 byte decoding

Output: A single **raw text string** passed downstream for skill extraction

Implementation: See **extractTextFromResume()** in **ai-engine/services/resume_processor.py**

2. Skill Extraction from Resume

Algorithm: **Dictionary-Based Pattern Matching** with **Regex** + Contextual Confidence Scoring

How it works:

1. Dictionary Lookup

- A predefined dictionary (**TECH_SKILLS**) maps ~150+ skill keywords (e.g., **"python"**, **"react"**, **"kubernetes"**) to categories:
 - **Language** (Python, JavaScript, Java, C++, etc.)
 - **Frontend** (React, Vue, Angular, HTML, CSS, etc.)
 - **Backend** (Node.js, Django, Flask, Spring, etc.)
 - **Database** (MongoDB, PostgreSQL, MySQL, Redis, etc.)
 - **Cloud** (AWS, Azure, GCP)
 - **DevOps** (Docker, Kubernetes, Jenkins, CI/CD, etc.)
 - **AI/ML** (TensorFlow, PyTorch, scikit-learn, etc.)
 - **Mobile** (React Native, Flutter, iOS, Android)
 - **Testing** (Jest, Pytest, Selenium, Cypress, etc.)

2. Pattern Matching

- For each skill in the dictionary, a **regex whole-word search** (**\b<skill>\b**) is run against the lowercased resume text
- This ensures exact matching (e.g., **"python"** won't match **"pythonic"**)

3. Confidence Scoring

- **Base score** = $\min(70 + \text{frequency} \times 5, 95)$ — more occurrences = higher confidence
- **Context boost** (+5 points) is applied if the skill appears near keywords like:
 - "experience"
 - "proficient"
 - "expert"
 - "skilled"
 - "worked with"
 - "developed"
- Context is checked within a 200-character window around the skill mention
- Confidence is capped at **98%**

4. Deduplication

- If a skill maps to multiple dictionary entries (e.g., "reactjs" and "react.js"), only the one with the highest confidence is kept
- Special name normalization handles cases like "javascript" → "JavaScript", "nodejs" → "Node.js"

5. Ranking & Output

- Skills are sorted by confidence (descending)
- The **top 20 skills** are returned

Implementation: See `extractSkillsUsingNLP()` in `ai-engine/services/resume_processor.py`

Note: spaCy (`en_core_web_sm`) is loaded optionally but currently unused. The system relies purely on pattern matching. The NLP model could be used for Named Entity Recognition (NER) in future versions.

3. Embedding Generation

Algorithm: Sentence-BERT (specifically the `all-MiniLM-L6-v2` model)

How it works:

- **Model:** From the `sentence-transformers` library — a fine-tuned variant of BERT optimized for producing semantically meaningful sentence embeddings
- **Architecture:** `all-MiniLM-L6-v2` is a lightweight 6-layer MiniLM model
 - **Parameters:** 22 million
 - **Output:** 384-dimensional dense vectors
 - **Training:** Trained on over 1 billion sentence pairs using a **contrastive learning** objective

Process:

1. Input text is tokenized using WordPiece tokenization
2. Tokens are passed through the 6-layer transformer encoder
3. A **mean pooling** layer averages all token embeddings into a single 384-dimensional vector
4. The vector is normalized (L2 normalization)

Two Types of Embeddings:

Skill Embedding:

- All user skills are joined into one string (e.g., "Python React Node.js MongoDB")
- Encoded into a 384-dim vector representing the user's skill profile

Repository Embedding:

- A combination of fields is concatenated:

```
repo_name + description + language + topics
```

- Example: "fastapi A modern, fast web framework for building APIs with Python python web framework rest api"
- Encoded into a 384-dim vector representing the repository's technical profile

Implementation: See `generateEmbeddings()` in `ai-engine/services/skill_matcher.py`

4. Matching Skills with Repositories

Algorithm: Cosine Similarity on Sentence-BERT embeddings + Heuristic Boosting

Step A — Candidate Retrieval (GitHub Search API)

1. The top 5 user skills are combined with OR logic
2. Filter `good-first-issues:>0` is added to ensure beginner-friendly repos
3. GitHub Search API query example:

```
python OR react OR nodejs OR mongodb OR docker good-first-issues:>0
```

4. Up to **50 repositories** are fetched, sorted by stars (descending)
5. **Large organization filtering:**
 - Repos owned by major tech companies (Microsoft, Google, Meta, Amazon, Netflix, etc.) are filtered out
 - This ensures recommendations favor **individual developers** and small open-source projects
 - ~80 organizations are in the exclusion list

Step B — Semantic Ranking (Cosine Similarity)

For each candidate repository, compute the similarity between skill embedding and repo embedding:

$$\text{similarity} = \frac{\vec{A} \cdot \vec{B}}{|\vec{A}| \times |\vec{B}|}$$

Where:

- $\vec{A}$ = Skill embedding (384-dim vector)

- ### Normalization:

- ## Boosting:

- ### Step C — Difficulty Classification (Heuristic)

Stars	Forks	Difficulty
> 50,000	> 10,000	Hard
> 10,000	> 2,000	Medium
Otherwise	—	Easy

Step D — Match Reason Generation

- ## Step E — Final Output

- Fallback:** If the Sentence-BERT model fails to load, a **simple keyword matching** fallback is used:

- Implementation:** See `getTopRepositories()` in `ai-engine/services/skill_matcher.py`

5. Contribution Guide Generation

Algorithm: LLM Prompt Engineering using **Google Gemini 2.5 Flash** (with template fallback)

How it works:

Step 1 — Prompt Construction

A structured prompt is built containing:

- **Repository metadata:** name, description, language, stars, topics
- **Issue details** (if available): number, title, labels, difficulty, comment count
- **User skills:** top 10 skills from their profile

Example prompt structure:

```
You are a helpful open-source contribution mentor. Generate a personalized contribution guide for a developer who wants to contribute to a specific issue.
```

```
Repository: username/project-name
Description: A modern web framework
Language: Python
Stars: 15000
Topics: python, web, framework, api, rest
```

```
Specific Issue to Contribute:
- Issue Number: #42
- Issue Title: Add authentication middleware
- Labels: good first issue, enhancement
- Difficulty: medium
- Comments: 3 comments
```

```
Developer's Skills: Python, Flask, REST, MongoDB, Docker
```

```
Please provide a JSON response with the following structure:
```

```
{
  "summary": "...",
  "issueAnalysis": {...},
  "gettingStarted": [...],
  "codeConventions": [...],
  "tips": [...]
}
```

Step 2 — LLM Generation

- **Model:** Google Gemini 2.5 Flash (fast, cost-effective variant)
- The Gemini API call is run in a **thread pool executor** (since the SDK is synchronous)
- This avoids blocking the async FastAPI event loop
- Response is parsed as JSON after stripping markdown code fences

Step 3 — Response Structure

The LLM returns a structured guide with:

Summary:

- 2-3 sentence overview of the issue and why it's a good contribution opportunity

Issue Analysis:

- Difficulty level (easy/medium/hard)
- Estimated time to complete (e.g., "3-5 hours")
- Skills needed for this specific issue

Getting Started:

- Step-by-step instructions (fork, clone, setup, etc.)
- Specific to the issue (references issue number in commit messages)

Code Conventions:

- Project-specific coding standards
- Best practices for the language/framework used

Tips:

- Communication advice (commenting on the issue)
- Review process expectations
- Community etiquette

Resources:

- Links to the issue, repository, and helpful guides

Step 4 — Template Fallback

If Gemini is unavailable (no API key, rate limit, error), a **template-based guide** is generated:

- Uses string interpolation with repo/issue metadata
- Produces hardcoded but contextual step-by-step instructions
- Still provides value with generic best practices

Implementation: See `generate()` and `generateGuideUsingLLM()` in `ai-engine/services/guide_generator.py`

Summary Table

Task	Algorithm / Model	Key Technique	File
Text Extraction	PDFMiner / python-docx	Binary-to-text parsing	resume_processor.py
Skill Extraction	Regex Pattern Matching	Dictionary lookup + contextual confidence scoring	resume_processor.py

Task	Algorithm / Model	Key Technique	File
Embedding Generation	Sentence-BERT (all-MiniLM-L6-v2)	Transformer encoder + mean pooling → 384-dim vectors	skill_matcher.py
Repo Matching	Cosine Similarity	$\frac{\text{vec}(A) \cdot \text{vec}(B)}{\ \text{vec}(A)\ \cdot \ \text{vec}(B)\ }$ on embeddings + heuristic boosting	skill_matcher.py
Difficulty Classification	Rule-based Heuristic	Stars/forks thresholds	skill_matcher.py
Guide Generation	Google Gemini 2.5 Flash LLM	Structured prompt → JSON response	guide_generator.py

Model Specifications

Sentence-BERT (all-MiniLM-L6-v2)

- **Type:** Sentence Transformer (BERT-based)
- **Layers:** 6 transformer layers
- **Parameters:** 22 million
- **Embedding Dimension:** 384
- **Context Length:** 256 tokens
- **Training Data:** 1+ billion sentence pairs
- **Training Objective:** Contrastive learning (sentence similarity)
- **Performance:**
 - Fast inference (~0.01s per sentence on CPU)
 - Good semantic similarity for short texts
 - Optimized for retrieval and clustering tasks

Google Gemini 2.5 Flash

- **Type:** Large Language Model (LLM)
- **Purpose:** Text generation, instruction following
- **Strengths:**
 - Fast response time (optimized for throughput)
 - Good JSON output formatting
 - Strong code understanding
 - Contextual reasoning
- **Usage in GitCompass:**
 - Personalized guide generation
 - Issue-specific advice synthesis
 - Natural language output

Future Enhancements

Potential Algorithm Improvements:

1. Skill Extraction:

- Add **Named Entity Recognition (NER)** using spaCy for better extraction
- Implement **context-aware skill categorization** (e.g., "Python for ML" vs "Python for web")
- Use **TF-IDF** or **keyword extraction** algorithms for discovering unlisted skills

2. Repository Matching:

- Incorporate **collaborative filtering** (recommend repos based on similar users)
- Add **temporal signals** (prioritize recently active repos)
- Implement **diversity ranking** to avoid recommending too many similar repos

3. Guide Generation:

- **Multi-agent LLM system** with specialized agents for different guide sections
- **RAG (Retrieval-Augmented Generation)** to fetch actual code examples from the repo
- **Fine-tuned model** specifically for open-source contribution guidance

4. Performance:

- **Vector database** (e.g., Pinecone, Weaviate) for faster similarity search at scale
- **Model quantization** for faster embedding generation
- **Caching layer** for frequently requested guides

Last updated: February 10, 2026